

INFO 510 Bayesian Modelling and Inference

Lecture 16 – Convergence Diagnostics in MCMC *(MCMC algorithm)*

Dr. Kunal Arekar
College of Information
University of Arizona, Tucson

December 1, 2025

Steps in a typical Bayesian analysis

Define the Model and Prior - Start with a likelihood function (based on your data) and choose prior distributions for the parameters

Run MCMC - Since the posterior distribution is usually complex and hard to compute directly, you use an MCMC algorithm (like Gibbs sampling or Metropolis-Hastings) to generate samples from this posterior distribution. MCMC helps draw samples in a way that approximates the true posterior shape without needing to solve it analytically

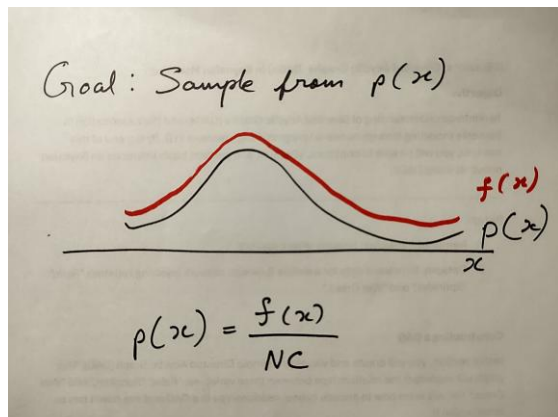
Analyze the Posterior Samples - Once you have a sufficient number of samples, you analyze them to estimate the posterior distribution — calculating means, medians, credible intervals, and other summary statistics

Chains in MCMC

MCMC chains are generated by different algorithms – e.g. Metropolis-Hastings and Gibbs sampling

Metropolis-Hastings

- Starts with an initial guess for θ
- Proposes a new value for θ based on a "proposal distribution"
- Decides whether to "accept" this new value or "reject" it, using a probability that depends on how likely the new value is according to the posterior
- If accepted, the new value is added to the chain; if rejected, the chain keeps the current value
- Over time, this process builds up samples around the high-probability areas of the posterior, but occasionally samples lower-probability areas, too, preventing the chain from getting stuck



The accept-reject step

calculate the acceptance probability, $A(x_n \rightarrow x^*)$,

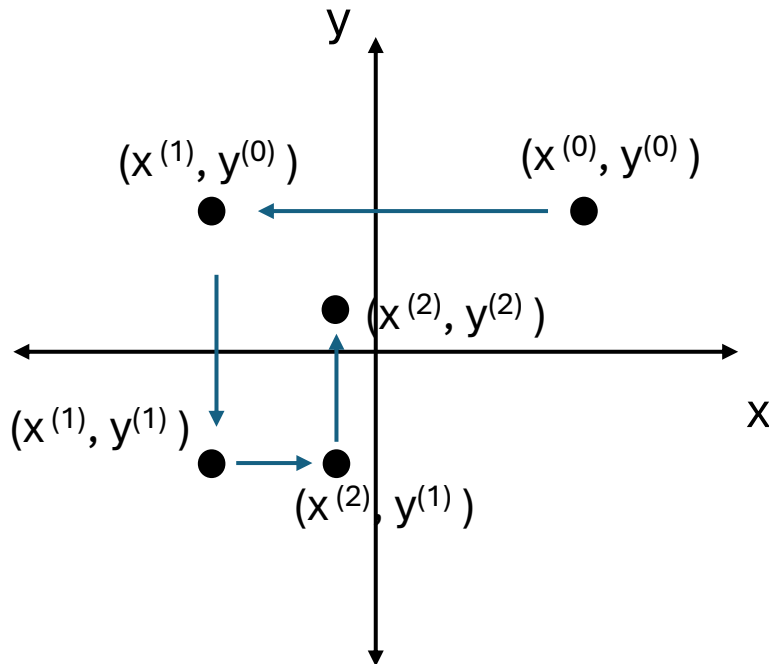
$$A(x_n \rightarrow x^*) = \min \left(1, \frac{p(x^*)}{p(x_n)} \times \frac{q(x_n|x^*)}{q(x^*|x_n)} \right)$$

Chains in MCMC

MCMC chains are generated by different algorithms – e.g. Metropolis-Hastings and Gibbs sampling

Gibbs Sampling

- Uses a simpler approach where we can sample each parameter conditionally
- For each parameter, it samples from its conditional posterior, fixing the values of all other parameters
- This sequential updating process (sampling one parameter at a time) is repeated until it creates a set of values approximating the posterior



Keep alternating between x and y , each time sampling from their conditional distributions

Eventually, the sequence of samples will converge to the true joint distribution

Convergence in MCMC

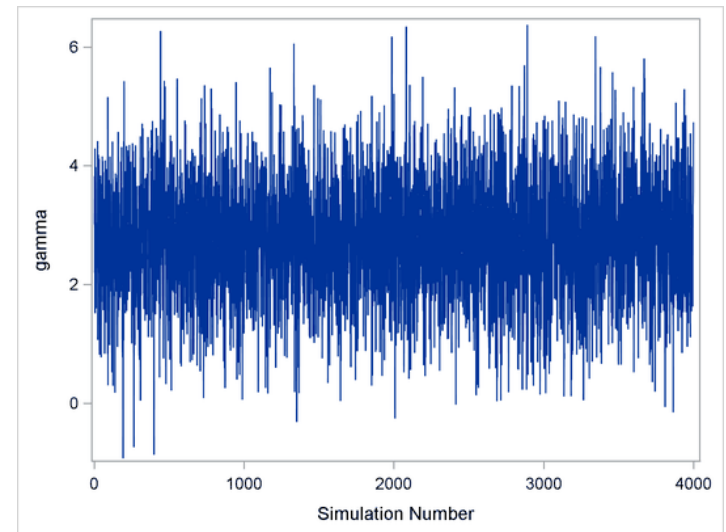
- From our theory of Markov chains, we expect our chains to eventually converge to the stationary distribution, which is also our target distribution.
- However, there is no guarantee that our chain has converged after M draws.
- How do we know whether our chain has actually converged?
- We can never be sure, but there are several tests we can do, both visual and statistical, to see if the chain appears to be converged.

Convergence in MCMC

convergence occurs when the chain has "settled" into a distribution that represents the target posterior, meaning the samples reflect the true properties of the posterior distribution

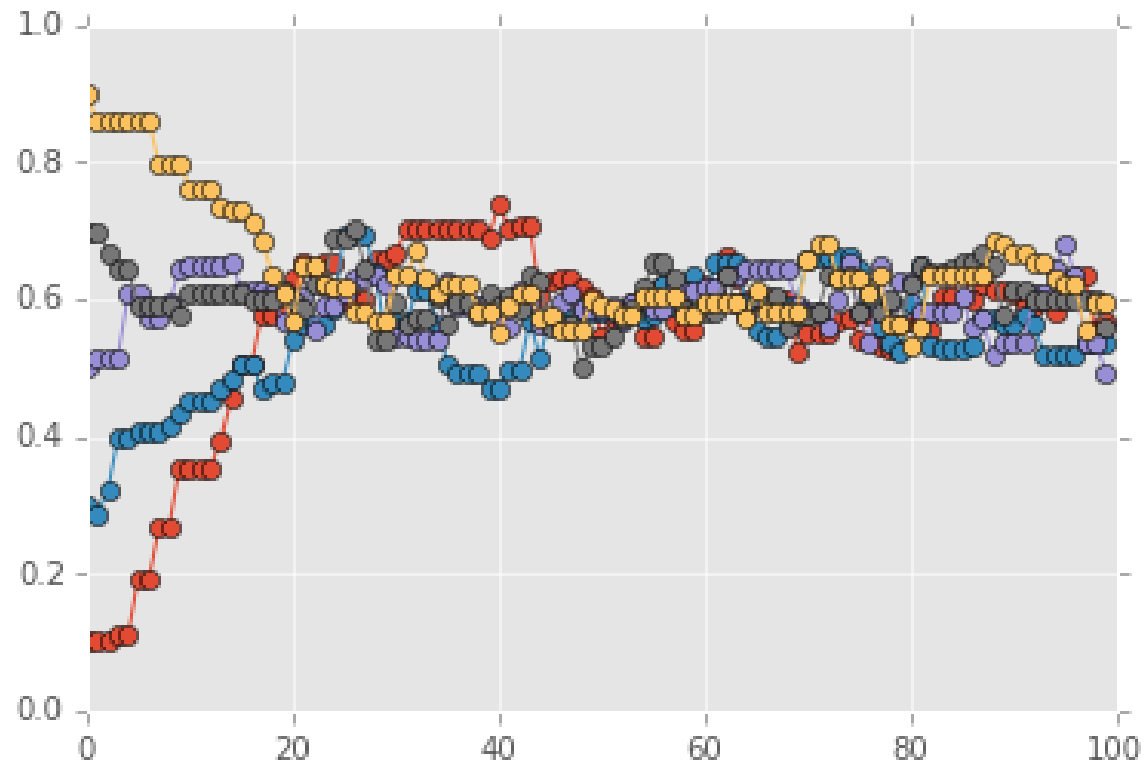
Convergence refers to the idea that, a MCMC technique that we choose will eventually reach a stationary distribution

From this point on it stays in this distribution and moves about (or mixes) throughout the subspace forever



Convergence in MCMC

MCMC sampling with 5 chains



Common MCMC Convergence Challenges

High-Dimensional Spaces - More dimensions can complicate convergence, as the chain must adequately explore each direction

Poorly Chosen Starting Values - Chains starting far from the high-probability regions may take longer to converge

Complex Posteriors - Multi-modal or heavy-tailed distributions may cause the chain to get "stuck" in one area

What kind of problems do MCMC diagnostics try to spot?

There are two main problems that we may encounter

Problem 1 –

A large portion of the sample is drawn from distributions that are significantly different from the target

Problem 2 –

The effective size of the sample (ESS) is too small

Problem 1 - A large portion of the sample is drawn from distributions that are significantly different from the target (Poor convergence/slow mixing)

As number of MCMC samples (t) tends to infinity, the distribution of x_t converges to the target distribution

If the chain is slow to converge, that is, after many iterations, the distributions of the draws are still very different from the target distribution

Typically, this happens when the initial value x_1 lies in a region to which the target distribution assigns a very small probability

A large portion of our MCMC sample might be made up of observations drawn from distributions that are significantly different from the target distribution

Solution:

discarding a large chunk of initial observations (burn-in sample)
increasing the number of iterations (i.e., the sample size)

Problem 2 - The effective size of the sample is too small

In a MCMC sample, two terms of the chain X_t and X_{t+n} are generally not independent, but they become nearly independent as n increases

Let n_0 be the smallest number such that X_t and X_{t+n_0} are considered practically independent; then the sample size T can be divided by n_0 to determine an "effective sample size" of T/n_0

This effective sample size T/n_0 represents the MCMC sample size as if it were independent observations

When dependence decays slowly (high n_0), the effective sample size becomes small, leading to a noisy or imprecise empirical approximation of the target distribution (A low ESS indicates poor mixing and unreliable estimates)

Solution:

Modifying the MCMC algorithm to reduce dependence (lower n_0),
Increasing the total iterations (increasing T)

Total sample size T - we collect 1,000 data points from the MCMC simulations

Identifying Dependence - Suppose that we observe a degree of dependence between consecutive samples

Lag and Independence with n_0 - After analyzing the correlation between samples, we find that if we skip every 10 samples (e.g., X_t , X_{t+10} , X_{t+20} , etc.), the samples start to look approximately independent of each other. This "gap" of 10 is what we call n_0 , the smallest lag that makes two samples nearly independent

$$\text{Effective Sample Size} = T / n_0 = 1000 / 10 = 100$$

This means, even though we collected 1,000 samples, our data behaves as if we only had 100 independent samples due to the dependence structure of the chain

A ESS lower than this number indicates poor mixing, suggesting potential convergence issues

And it also means our sample doesn't capture the target distribution as well as a truly independent sample would, therefore making it less reliable

Convergence Diagnostics

Visual Diagnostic Tools	Trace plots
	Autocorrelation plots
Quantitative Diagnostic Tools	Gelman-Rubin (R-hat)
	Geweke
	Heidelberger-Welch (stationarity test)
	Heidelberger-Welch (half-width test)
	Raftery-Lewis
	effective sample size

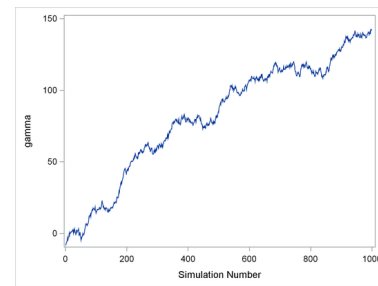
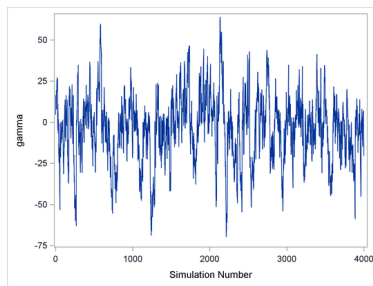
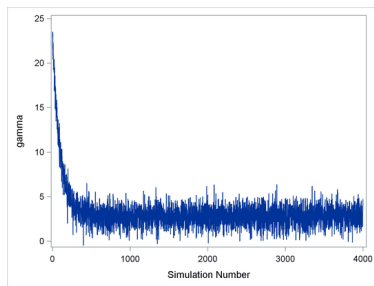
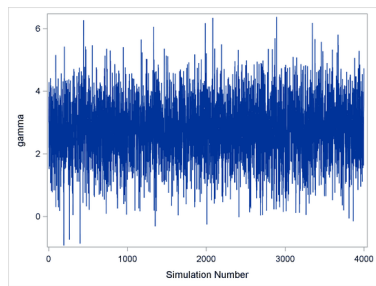
Trace Plots

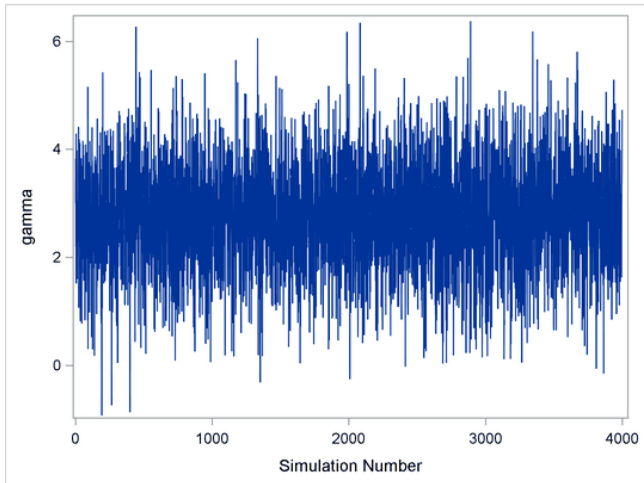
A trace plot displays sampled values against the iteration number, showing how the chain explores the parameter space

The trace tells you if the chain has not yet converged to its stationary distribution

A trace can also tell you whether the chain is mixing well

The aspects of stationarity that are most recognizable from a trace plot are a relatively constant mean and variance

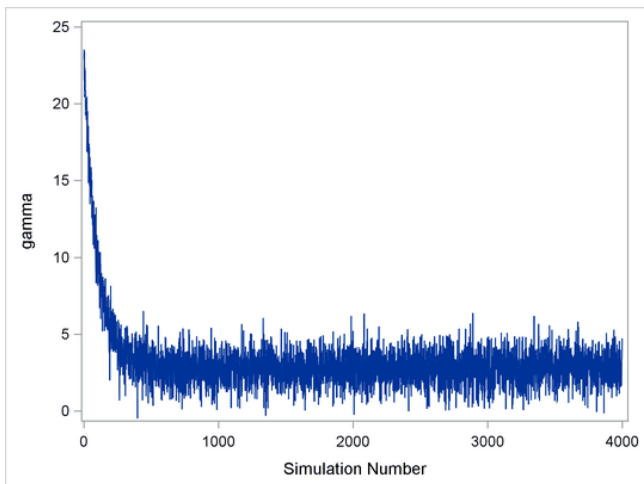




displays a perfect trace plot

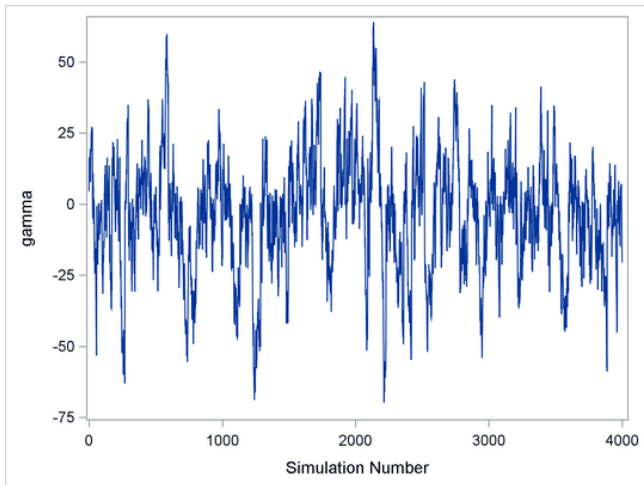
chain appears to be around the value 3, with very small fluctuations

this indicates that the chain could have reached the right distribution



displays a trace plot for a chain that starts at a very remote initial value and makes its way to the targeting distribution

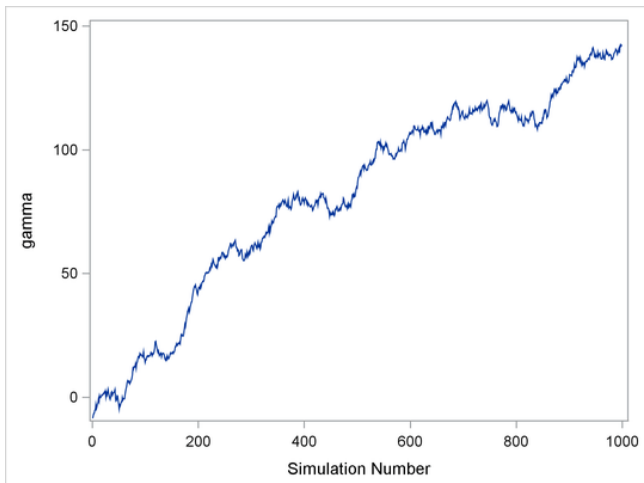
It travels relatively quickly to the target distribution, reaching it in a few hundred iterations



Demonstrates marginal mixing

This type of trace plot is typically associated with high autocorrelation among the samples

To obtain a few thousand independent samples, you need to run the chain for much longer



depicts a chain with serious problems

mixing very slowly, and offers no evidence of convergence

try to improve the mixing of this chain

Autocorrelation Diagnostic

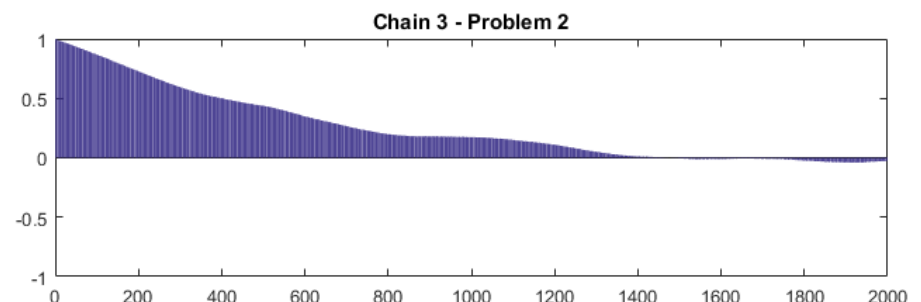
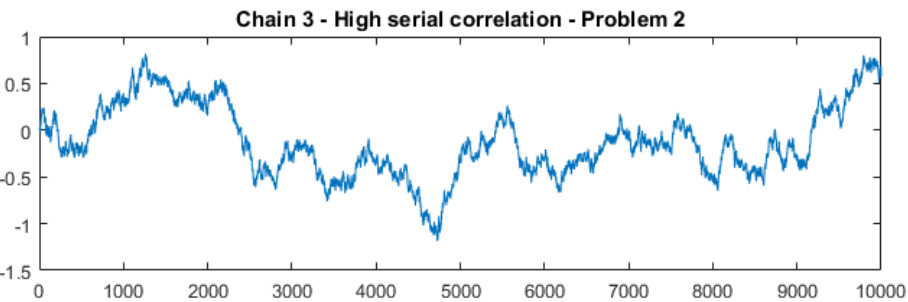
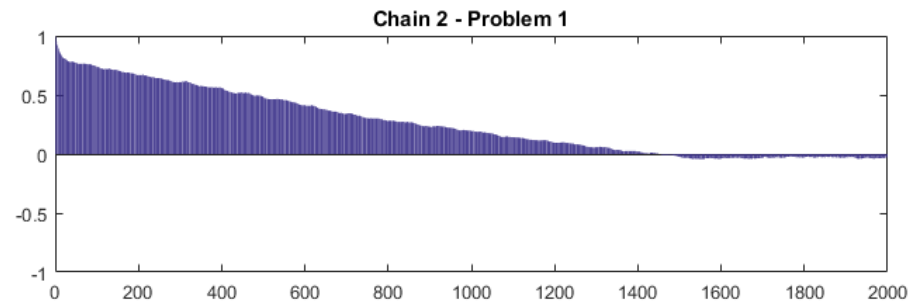
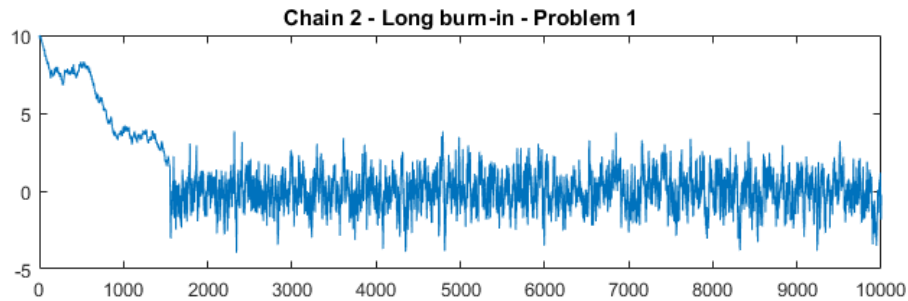
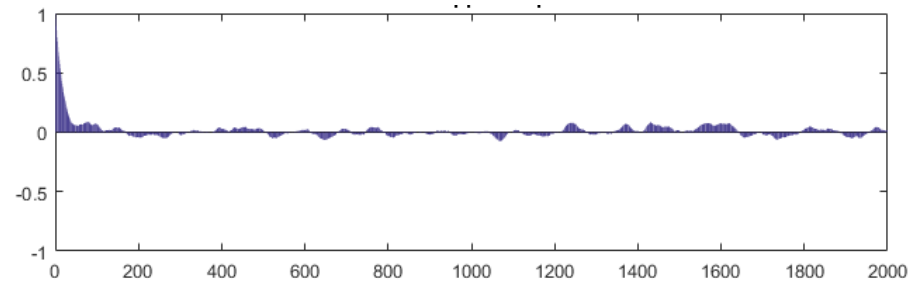
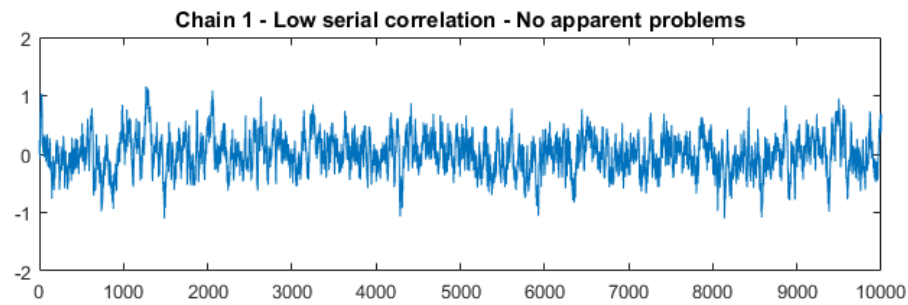
Autocorrelation refers to a pattern of serial correlation in the chain, where sequential draws of a parameter, say β_1 , from the conditional distribution are correlated.

The cause of autocorrelation is that the parameters in our model may be highly correlated, so the Gibbs Sampler will be slow to explore the entire posterior distribution.

The reason why autocorrelation is important is that it will take a very long time to explore the entire posterior distribution.

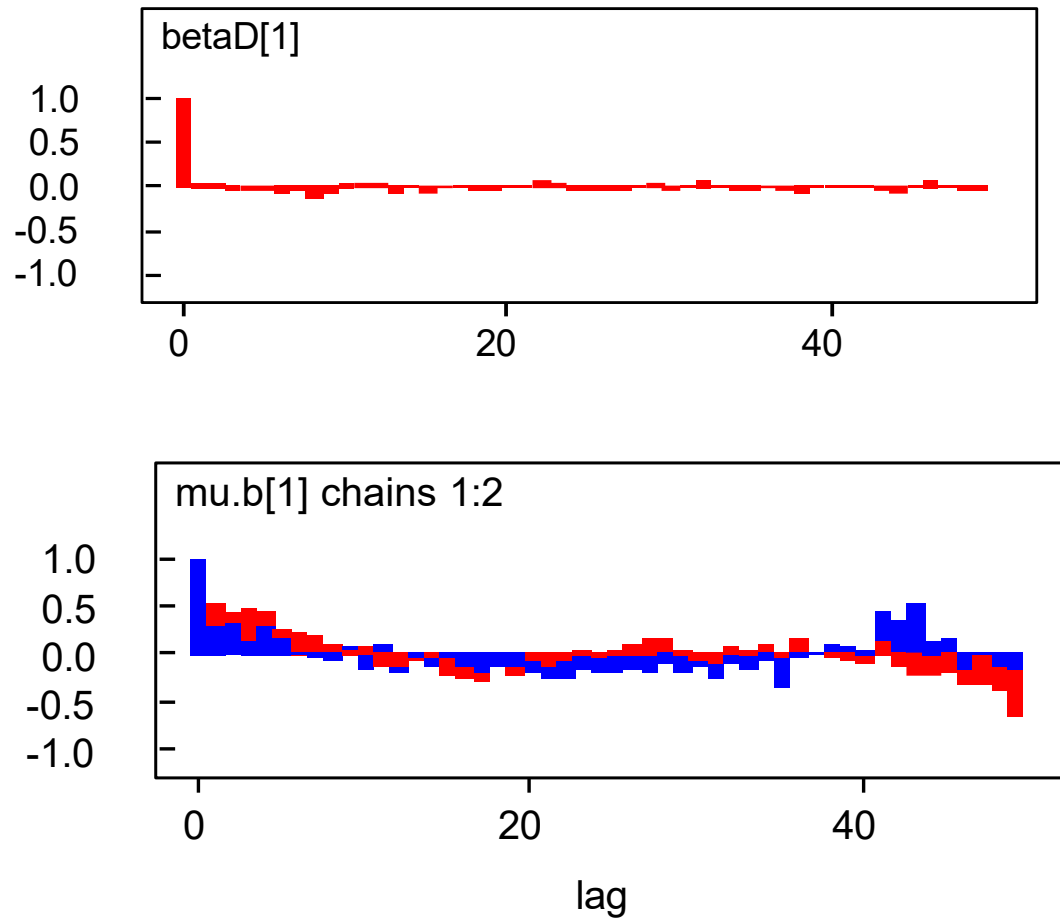
- Note that if the level of autocorrelation is high for a parameter of interest, then a traceplot will be a poor diagnostic for convergence.

Typically, the level of autocorrelation will decline with an increasing number of lags in the chain (e.g. as we go from the 1000th to the 1010th lags, the level of autocorrelation will often decline.) When this dampening doesn't occur, then you have a problem and will probably want to re-parameterize the model



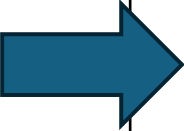
The plots of Chains 2 and 3 show that not only autocorrelation is large at short lags, but it also dies out very slowly

Example of Model without autocorrelation (top) and with autocorrelation (bottom)



In serious (and not uncommon cases), the autocorrelation plot will be a solid bar across the screen.

Convergence Diagnostics

Visual Diagnostic Tools	Trace plots
	Autocorrelation plots
 Quantitative Diagnostic Tools	Gelman-Rubin (R-hat)
	Geweke
	Heidelberger-Welch (stationarity test)
	Heidelberger-Welch (half-width test)
	Raftery-Lewis
	effective sample size

Gelman-Rubin Diagnostic (R-hat)

- compares the variance within a single chain to the variance between multiple chains, aiming for values of R-hat close to 1 as a sign of convergence
- test based on 2 or more parallel chains , (each started from different initial values).
- Best results are obtained for parameters whose marginal posterior densities are approximately normal
- This test can be computationally costly, because it prolongs the simulation M-fold; best to choose different initial values for all M chains
- initial values should be as dispersed from each other as possible so that the Markov chains can fully explore different parts of the distribution before they converge to the target
- This method cannot detect chain stuck in local maxima

Gelman-Rubin Diagnostic (R-hat)

Within chain variance

$$W = \frac{1}{M} \sum_{j=1}^M s_j^2$$

Between chain variance

$$B = \frac{N}{M-1} \sum_{j=1}^M (\bar{\theta}_j - \bar{\bar{\theta}})^2$$

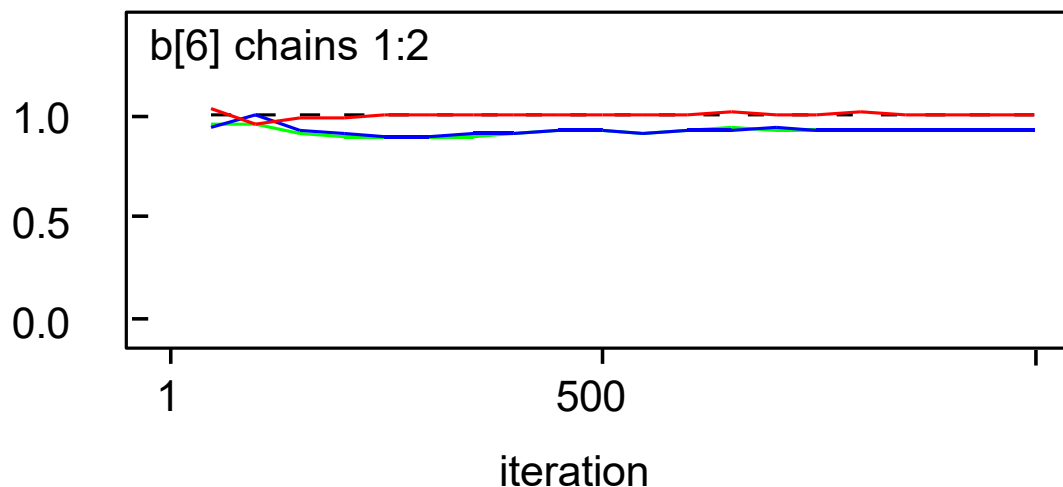
estimated variance of θ as the weighted sum
of between and within chain variance

$$\hat{var}(\theta) = \left(1 - \frac{1}{N}\right)W + \frac{1}{N}B$$

calculate the potential scale reduction factor

$$\hat{R} = \sqrt{\frac{\hat{var}(\theta)}{W}}$$

Example Diagnostic Plot for Gelman-Rubin consistent with convergence



The normalized width of the central 80% interval of the pooled runs is green

The normalized average width of the 80% intervals within the individual runs is blue

R is red. R would generally be expected to be greater than 1 if the starting values are suitably over-dispersed.

Brooks and Gelman (1998) emphasize that one should be concerned both with convergence of R to 1, and with convergence of both the pooled and within interval widths to stability.

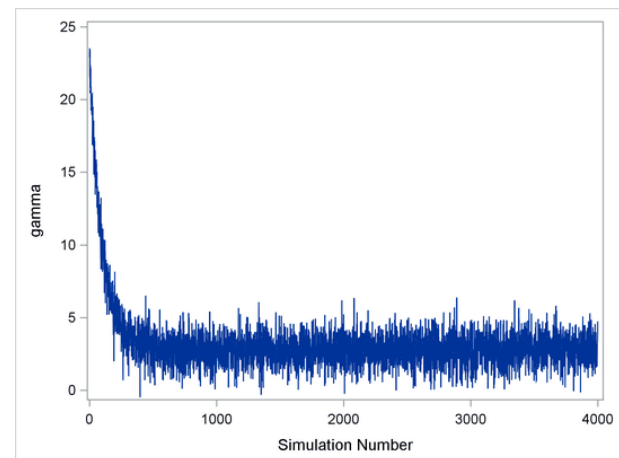
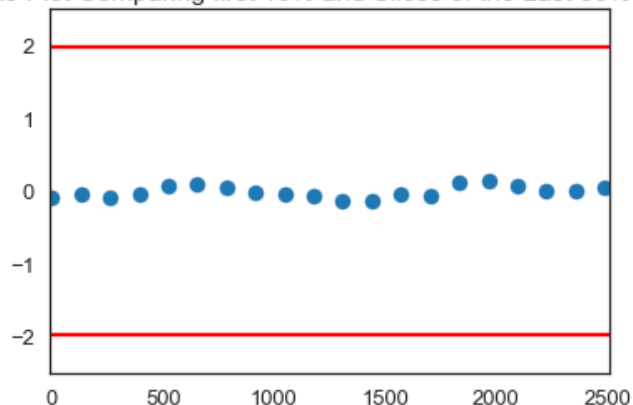
Geweke Time-Series Diagnostic

The Geweke Time-Series Diagnostic: if a model has converged, then if we simulate a large number of draws, the mean (and variance) of a parameter's posterior distribution from the first half of the chain will be equal to the mean (and variance) from the second half of the chain.

However, a perfectly reasonable way to proceed is look to see whether the posterior means (and variances) of your parameters are approximately the same for different halves of your simulated chain.

Geweke's diagnostic can be used to determine the burn-in period

Geweke Plot Comparing first 10% and Slices of the Last 50% of Chain



Geweke Time-Series Diagnostic

The diagnostic divides the MCMC chain into two non-overlapping portions –

Early portion: ~ first 10% of the chain

Late portion: ~ last 50% of the chain

For each parameter being estimated, the test calculates the mean of that parameter in each portion. Since MCMC draws are correlated rather than independent, the diagnostic accounts for this autocorrelation when computing the standard error of each mean.

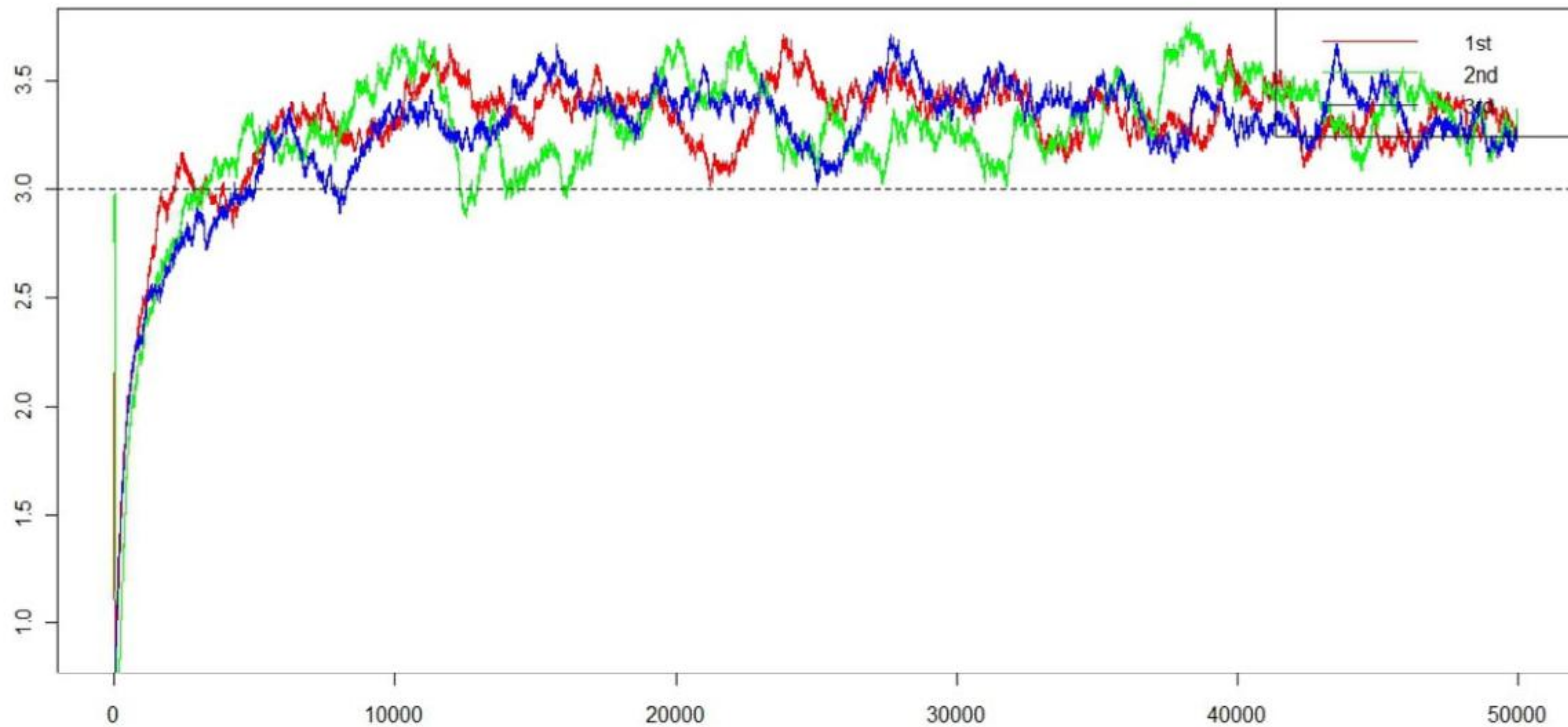
It then computes a test statistic:

$$Z = (\text{mean_early} - \text{mean_late}) / \text{SE}(\text{mean_early} - \text{mean_late})$$

This Z-statistic is compared against a standard normal distribution. If the chain has converged, this statistic should be small, typically with a p-value greater than 0.05, indicating no significant difference between early and late means.

Geweke Time-Series Diagnostic

Geweke's diagnostic can be used to determine the burn-in period



A typical MCMC run

Geweke Time-Series Diagnostic

Can we treat, say, the first 10,000 iterations as burn -in?

In a long -enough chain whose trace plot suggests convergence to the target distribution, we assume the second half of the chain has converged to the target distribution, and we test if the first 10% can be treated as burn -in.

mimic the simple two -sample test of means. If the mean of the first 10% is not significantly different from the last 50%, then we conclude the target distribution converged somewhere in the first 10% of the chain.

If the p-values from the Z tests are greater than 0.05 (significantly different), we may treat the first 10% as burn-in

Heidelberger-Welch

It tests the null hypothesis that the sampled values for each variable are form a stationary process.

If the null hypothesis is rejected for a given variable, the test is repeated after discarding the first 10% of iterations.

If the hypothesis is again rejected, a further 10% of iterations are discarded.

This process is repeated until either a portion of the chain (of length of the total number of iterations) passes the stationarity test, or 50% of the iterations have been discarded and the null hypothesis is still rejected.

If the latter occurs, This indicates that a longer run is needed in order to achieve convergence.

The Heidelberger-Welch diagnostic does this in two main steps

Burn-in Test (Stationarity Test): In MCMC, at the beginning of sampling, the samples are often chaotic and haven't yet settled down.

The Heidelberger-Welch diagnostic tests the first part of the sample (called the burn-in period) to see if the numbers are still messy or if they're starting to even out.

If it is messy, it discards the earlier samples and starts looking from a later point where the sample is smoother.

$$CvM = (1/n) \sum_{i=1}^n [\theta(i) - \bar{\theta}]^2$$

The Cramer-von Mises test statistic is based on how much the cumulative mean deviates from the overall mean across the entire chain

$$S_0 = (1/2\pi) \sum_{k=-\infty}^{\infty} \gamma(k)$$

Standardization to account for the correlation structure

$$Z = CvM / S_0$$

Test statistic

Common thresholds are $Z > 0.233$ (10% significance) or $Z > 0.461$ (1% significance)

If the test rejects stationarity, the algorithm removes the first 10% of the chain and repeats.

The Heidelberger-Welch diagnostic does this in two main steps

Convergence Test: Once the burn-in period is passed, the diagnostic checks if the rest of the sample is consistent.

If the sample is steady and consistent across different parts, the diagnostic says, “Okay, this sample is good to use for analysis.” If not, it suggests that you need to keep running the MCMC process longer.

Effective Sample Size and Standard Error $n_{\text{eff}} = n / (1 + 2\sum_{k=1}^{\infty} \rho(k))$

$$SE(\theta) = \sqrt{S_0 / n}$$

The "half-width" is the margin of error

$$HW = z_{\{\alpha/2\}} \times SE(\theta)$$

The test checks whether the half-width is sufficiently small relative to the mean estimate.

If the Heidelberger-Welch diagnostic passes both tests, it means the MCMC simulation has likely settled into a reliable pattern, and we can trust the results.

If it fails, then the simulation might need more iterations (more stirring) to reach a stable state.

Raftery-Lewis

This method applies to single chains

It tells us how big to make N based on our needs, and how much burn-in to throw away.

It is intended both to detect convergence to the stationary distribution and to provide bounds for the accuracy of the estimated quantiles of functions of variables of interest.

The user must specify the quantile to be estimated (the default is the 2.5th percentile), the desired degree of accuracy for the estimate of this quantile (the default is 0.005) and the required probability of attaining this degree of accuracy (the default is 0.95).

The output then reports $N_{\min}$ -- the minimum number of iterations that would be needed to estimate the specified quantile to the desired precision if the samples in the chain were independent.

To use R&L, we supply it with 4 numbers: Q , R , S , A

Q : a quantile that we want to estimate.

R : we want to estimate Q with the precision R .

S : the $S\%$ probability interval associated with the precision R .

A : a convergence tolerance that is used in determining how much burn-in to discard.

Burn-in is found as the minimum number of iterations of $B(i)$ it takes for the chain $B(i)$ to approach within A of its estimated stationary distribution.

N_{min} , the number of samples we need to estimate our quantile Q with the described precision R at the level S is found using binomial theory on the chain $B(i)$.

Given your quantile target q , precision r , and confidence level s , Raftery-Lewis tells us:

Output	Meaning
M	Total number of samples needed
N	Number of <i>independent</i> samples you'd need (in ideal, uncorrelated case)
k	Thinning interval (keep every k th sample)
Burn-in	Number of initial samples to discard
Dependence factor	How bad autocorrelation is (ratio of M/N)

This helps you to know -

How bad is my chain's autocorrelation?

Do I need more samples?

How much burn-in and thinning is recommended?

Convergence Diagnostic Summary

- 1) You can never prove that something has converged, you can only tell when something has not converged.
- 2) If your model has not converged and you are confident that YOU haven't made a stupid mistake, then the best thing to do may be to just let the model run a long time. CPU time is often "cheaper" than your time.

For models with large numbers of parameters you should let the model run for a long time.
- 3) There are a number of easy to implement tricks (mostly reparameterizations) that will help to speed convergence in most regression-based models.
- 4) Convergence does not mean that you have a good model!!!
Convergence should be the beginning of model assessment, not its end.

Tricks to speed convergence

Standardize all of your variables by subtracting them from their sample means (centering) and dividing by their sample variances (scaling).

- This speeds convergence by decreasing the posterior correlation between parameters.

In a simple example(linear model),

suppose that $Y_i \sim N(a + bX_i, 1)$ and that we chose flat priors for a and b . The posterior correlation between a and b is:

$$\rho_{a,b} = -\frac{\bar{x}}{\sqrt{\bar{x}^2 + \text{Var}(x)}}$$

If the absolute value of $\text{mean}(x)$ is large relative to the sample variance of x , then you will have a large posterior correlation between a and b and therefore slow convergence (due to a high autocorrelation in the parameter simulations).

$$\bar{x} \gg \text{Var}(x), \rho \rightarrow -1$$

Tricks to speed convergence

If you have multiple indicators of the same concept, create an index or use a latent variable model to decrease correlation among the predictors.

Tricks to speed convergence

Use multivariate (normal) priors.

Multivariate normal priors allow for prior correlation among regression coefficients.

```
model {  
  for (i in 1:15) {  
    DemDim1[i] ~ dnorm(muD[i], tauD)  
    muD[i] <- betaD[1] + betaD[2]*((DemActId[i])) + betaD[3]*((DemNotActId[i]))  
  }  
}
```

```
betaD[1:3] ~ dmnorm( nuD[], v1[ , ] )
```

```
nuD[1] <- -.687  
nuD[2] <- .099  
nuD[3] <- .212
```

Instead of placing separate priors

$$\beta_1, \beta_2, \beta_3 \sim \mathcal{N}(0, 1000)$$

```
v1[1,1] <- .1; v1[1,2] <- 0; v1[1,3] <- 0  
v1[2,1] <- 0; v1[2,2] <- .1; v1[2,3] <- 0  
v1[3,1] <- 0; v1[3,2] <- 0; v1[3,3] <- 0.1
```

Use multivariate prior

$$\beta \sim \mathcal{N}(\mu, \Sigma)$$

```
tauD ~ dchisqr(1)  
}
```

Tricks to speed convergence

Pick good initial values.

If your initial values are near their posterior modes, then convergence should occur relatively quickly, especially if there is not an autocorrelation problem.

Tricks to speed convergence

Change sampling algorithms.

This can be a little tricky, but can really speed up the chains and help with mixing.

It requires you to go into the ‘guts’ of the Bugs files and change some default settings.

It’s not hard, but you can certainly make some mistakes here and mess a lot of things up— potentially.

So BE CAREFUL!

Tricks to speed convergence

Just wait: Sometimes models just take a long time to converge.

In extreme cases, it may take 1000000 iterations of your sampler to fully explore the posterior distribution.

Unfortunately, this tends to occur in models with a large number of parameters that are already slow to run (especially if you also have a large n).

The problem with this is that storing 1000000 samples * m parameters requires considerable memory. This may become particularly important if you try to calculate multiple sample statistics at once after convergence.

A mildly controversial solution to this is to “thin” your chain, by only storing every k th sample of your chain.

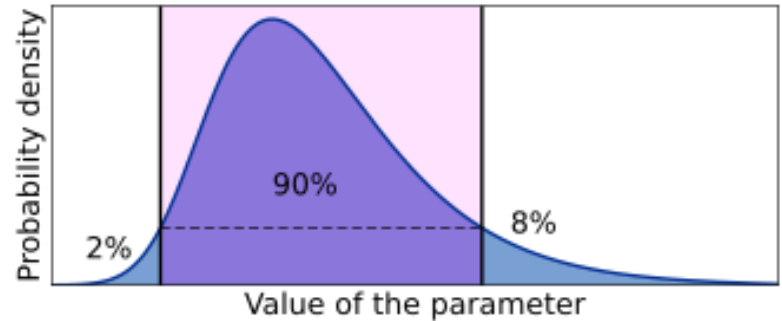
Summary of all the tricks

Trick	Goal	Analogy
Standardize variables	Reduce correlation	Level the playing field
Index/latent variable	Reduce redundancy	Fewer compasses
Multivariate priors	Incorporate structure	Adjust for wind while aiming
Over-relaxation	Improve mixing	Pendulum with extra push
Good initial values	Reduce burn-in	Start halfway up the trail
Change algorithms	Adapt to geometry	Switch vehicle
Just wait / thin	Let model converge	Let the soup simmer

Credible Intervals

Posterior probability that the parameter lies within the interval

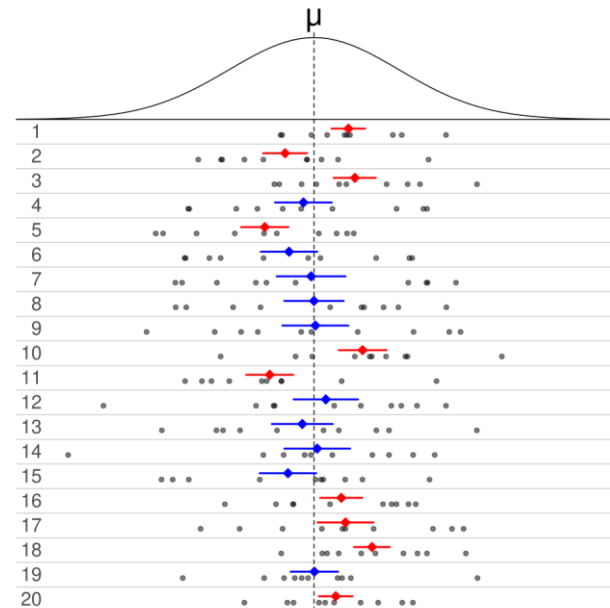
Given the data and prior beliefs, there's a specific probability that the true value of the parameter is within this interval



Frequentist Confidence Interval

The fraction of intervals calculated from a large number of data sets generated by the same process that would include the true parameter value

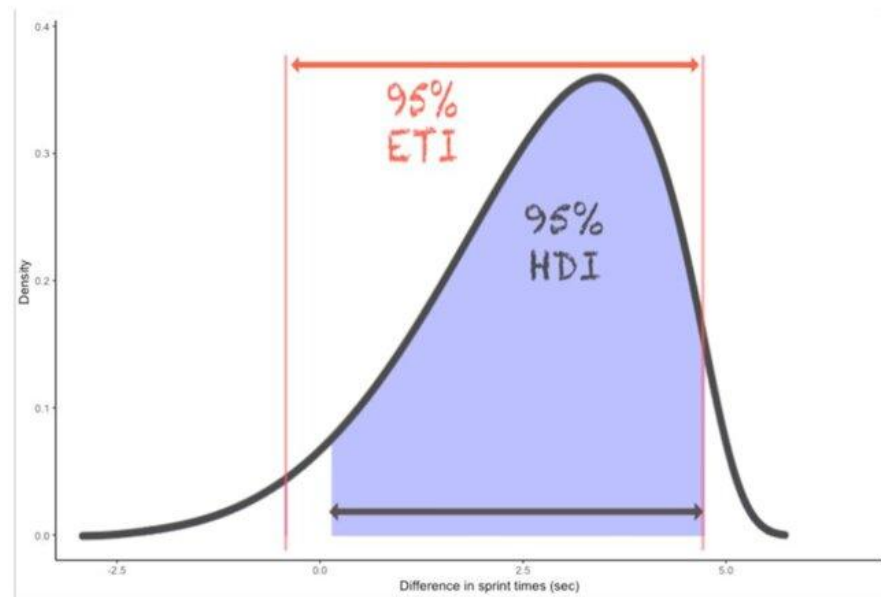
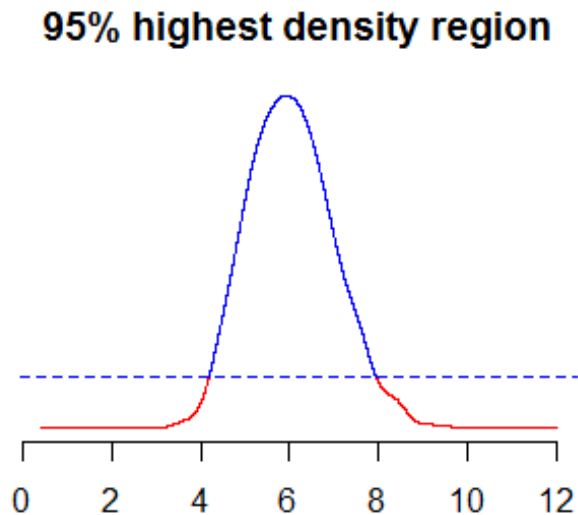
if you repeat an experiment many times, a certain percentage (e.g., 95%) of those intervals would contain the true parameter



Types of Credible Intervals

Highest Posterior Density (HPD) Interval - The shortest interval that contains 95% of the posterior probability. It includes all points where the posterior density is higher than outside the interval

Equal-Tailed Interval - Includes 2.5% of the distribution in each tail, ensuring that 95% of the distribution lies between the two endpoints



Credible Intervals

$$\int_{-\infty}^A p(\theta|Y) d\theta + \int_B^{\infty} p(\theta|Y) d\theta = \alpha$$

- Analytically estimated from posterior CDF
- Numerically estimated from quantiles of sample
- NOT estimated based on standard deviation
- Not necessarily symmetric
- Equal tail interval: both tails have the same probability
- Highest posterior density: narrowest possible interval